



Windows Scripting Host

More power to the double-click!

Mayur Bhatia

Few are aware of scripting with batch files, but people don't use batch files much unless when working with mountainous tasks. Even fewer are aware of Windows Scripting Host (WSH), a scripting environment that comes integrated with Windows to help automate your tasks, and what's more, unlike batch scripting, WSH has very few restrictions; you can create scripts to create, edit, and delete text files, Registry entries, create users, change passwords... you name it.

Scripts alone can be so powerful that one can even create viruses using it: the dreaded ILOVEYOU virus is, at its core, just a text file containing a script. Don't despair—creating and executing scripts in Windows doesn't require you to install any software or compile anything; instead, it requires code—a lot of code. OK, despair a bit, but in the end it'll be worth it.

To be clear, first and foremost, this is not a tutorial to WSH itself, but rather a collection of scripts for automating certain tasks. So if anyone wants to jump straight to creating and using the script, you can simply type it in and save it as a file. For those hungry for detail, we'll be explaining each script as it comes. Hopefully, this should be help enough for you to start writing your own scripts.

About the only thing you will need to create and run the scripts is the trusty Notepad. Enter the script text and save the file with a .vbs extension. Whenever you want to use the script, simply double-click the file to execute your script. That said, let's get down to the scripts.

Start multiple programs and documents at the same time

This one will allow you to start several programs and open several documents upon opening a single file. You can start any number of programs you want in this manner. Individual files will open as if you'd double-clicked on them. This means the files will open the program associated with them. Remember that this function, at its core, is a simple Run command. Anyhow, here it is:

```
Set WshShell =
```

```
WScript.CreateObject("WScript.Shell")
WshShell.Run "Notepad"
WshShell.Run "C:\Reports\Reports.doc"
WshShell.Run
WshShell.Run "C:\Presentation\Presentation.ppt"
WshShell.Run
WshShell.Run "C:\Presentation\Presentation.pps"
WshShell.Run "www.google.com"
WshShell.Run "C:\MP3\Dragula.mp3"
```

Simple, isn't it? This one uses only a single function—"run". The line at the top indicates we want access to the WScript.Shell object. The following lines, like they suggest, tell Windows that we want to use the Run function. Remember that these programs will open in order from top to bottom, and you can enter as many run functions as you want. This means you will be able to open your Word file, your presentation document, Notepad, and also browse to your favourite site, all the while listening to an MP3—all with nothing but a simple double-click of a file. No more hunting around for different icons!

Take care not to make little mistakes when it comes to the spaces, quotes, commas, and dots, though the script text is not case sensitive.

Copy and move files and folders

On we come to the main part. This is what you will find yourself using the most. Performing daily tasks like copying / moving files and folders to and from one folder to another can be a painful task. Not anymore, not when you have this script at your disposal:

```
Set FSO =
CreateObject("Scripting.FileSystemObject")
FSO.CopyFile "C:\text\*.\"", "C:\text1"
FSO.MoveFile "C:\test\*.\"", "C:\test1"
```

The first line declares the filesystem object, since we want to make changes to the filesystem itself. FSO.CopyFile and FSO.MoveFile are the functions used to copy and move files respectively. In this case it's all files in the folders "text" and "test"; the *.* characters indicates we want to copy all files from their specified directories on to the destination, whose path is declared at the end. Again, you can use these commands however many times you want in the script for multiple tasks.

Delete files

You probably saw this coming after the last one: you can use the "deletefile" object to delete any files. This is useful when you want to make sure that certain directories are empty or certain files are deleted before you shut down Windows, or simply to clear out all temporary files. The script:

```
Set FSO =
CreateObject("Scripting.FileSystemObject")
FSO.DeleteFile "C:\test1\temp.doc"
```

Rather self-explanatory. You're already familiar with the first part by now; the next line has an easy-to-understand function, "FSO.DeleteFile", followed by the path to the folder or file you want to delete. If you want to nuke the entire directory, use the *.* variable to include all files in the directory specified in the script.

Map a network drive using a shortcut

Ever wanted a simple shortcut to map your network shares to a drive? You can do so using this script. Remember that you can also add as many drives you want to map, similar to WshShell.Run. Here it is:

```
Set WSHNetwork =
WScript.CreateObject("WScript.Network")
WSHNetwork.MapNetworkDrive
"S:", "\\network\share 1"
WSHNetwork.MapNetworkDrive
"T:", "\\network\share 2"
WSHNetwork.MapNetworkDrive
"U:", "\\network 2\share 3"
```

Again, the first line asks WSH to declare the "WSHNetwork" object. As mentioned before, since we are making changes to the network subsystem—a network drive in this case—we'll need to declare it, like we have done in the first line. Further down, we see "WSHNetwork" using the "MapNetworkDrive" function, this is the actual command for creating a network drive. Take note that the drives themselves are in their own set of quotes followed by the network shares. Similar to the Run function earlier, the drives will be mapped in order from top to bottom.

Disconnect network drives

Yes, similar to how you can map your network drives together, you can also